

```

1  //////////////////////////////////
2  // CLASSIFIER.H //
3  //////////////////////////////////
4
5  /* UNIFICATION OF LBPFeatures + SVM Blocks */
6
7  #ifndef CLASSIFIER_H
8  #define CLASSIFIER_H
9
10 #include <systemC.h>
11 #include <cstdint>
12
13 #define IMG_WIDTH 28
14 #define IMG_HEIGHT 28
15 #define HISTOGRAM_WIDTH 256
16 #define SPLIT 2
17 #define TYPES 10
18
19
20 SC_MODULE(Classifier) {
21     sc_in<bool> clk;
22     sc_in<bool> reset;
23     sc_in<bool> start;
24     sc_in<uint8_t> pixels[IMG_WIDTH * IMG_HEIGHT];
25     sc_out<uint8_t> prediction;
26
27
28     SC_CTOR(Classifier) {
29         SC_THREAD(classify);
30         sensitive << clk.pos();
31         async_reset_signal_is(reset, true); //
32     }
33
34     void classify();
35
36     void LBPFeatures(const sc_in<uint8_t> pixels[IMG_HEIGHT * IMG_WIDTH], float feature[HISTOGRAM_WIDTH]);
37
38     void SVM(float features[HISTOGRAM_WIDTH], uint8_t *prediction);
39
40     float fast_sqrt(float number);
41 };
42
43 #endif // CLASSIFIER_H

```

```

1  //////////////////////////////////
2  // CLASSIFIER.CPP //
3  //////////////////////////////////
4
5  #include "classifier.h"
6  #include "model.h" //Introduce SVM_Model parameters
7  #include <cmath>
8
9  void Classifier::classify() {
10     uint8_t temp;
11     while (true) {
12         wait();
13
14         //Synchronous behavior
15         if (start.read()) {
16             float features[HISTOGRAM_WIDTH];
17
18             LBPFeatures(pixels, features);
19
20             SVM(features, &temp);
21
22             prediction.write(temp);
23         }
24     }
25 }
26
27 void extract_histogram(uint8_t pixels[IMG_HEIGHT * IMG_WIDTH / SPLIT + 2 * IMG_WIDTH], uint16_t histogram[HISTOGRAM_WIDTH], uint8_t length){
28
29     uint8_t transform;
30     uint8_t i, j, kk;
31
32     loop_height: for (i = 1; i < length; i++){
33         loop_width: for (j = 1; j < IMG_WIDTH - 1; j++){
34
35             transform = 0;
36             loop_neighbourhood: for(kk = 0; kk < 8; kk++){
37
38                 // Associate offset according to neighbourhood position
39                 int8_t row_offset = (kk == 1 || kk == 2 || kk == 3) ? -1 : (kk == 5 || kk == 6 || kk == 7) ? 1 : 0;
40                 int8_t col_offset = (kk == 3 || kk == 4 || kk == 5) ? -1 : (kk == 7 || kk == 0 || kk == 1) ? 1 : 0;
41
42                 // Add the correspondant binary weight if the neighbor is higher
43                 transform += (pixels[i * IMG_WIDTH + j] < pixels[(i + row_offset) * IMG_WIDTH + (j + col_offset)]) ? (1 << kk) : 0;
44
45             }
46
47             histogram[transform]++;
48         }
49     }
50 }
51
52 void Classifier::LBPFeatures(const sc_in<uint8_t> pixels[IMG_HEIGHT * IMG_WIDTH], float feature[HISTOGRAM_WIDTH]) {
53     uint16_t i;
54     uint8_t j;
55
56     uint16_t histogram[SPLIT][HISTOGRAM_WIDTH];
57     uint8_t slice_pixels[SPLIT][IMG_HEIGHT*IMG_WIDTH/SPLIT+2*IMG_WIDTH];
58     uint32_t histogram_product[HISTOGRAM_WIDTH];
59     uint32_t acum = 0;
60
61     // Initialise histogram to 0
62     loop_ini_hist2: for (i = 0; i < HISTOGRAM_WIDTH; i++) {
63         loop_ini_hist1: for (j = 0; j < SPLIT; j++) {
64             histogram[j][i] = 0;
65         }
66     }
67
68     uint16_t slice_size = IMG_HEIGHT * IMG_WIDTH / SPLIT;
69     uint16_t extended_slice_size = slice_size + 2 * IMG_WIDTH;
70
71     // Split pixels
72     loop_split2: for (i = 0; i < extended_slice_size; i++) {
73         loop_split1: for (j = 0; j < SPLIT; j++) {
74             if (i < (j == SPLIT - 1) ? slice_size : extended_slice_size) {
75                 slice_pixels[j][i] = pixels[i + j * slice_size];
76             }
77         }
78     }
79
80     // Obtain histogram from sliced pixels
81     loop_process: for (j = 0; j < SPLIT; j++) {
82         extract_histogram(slice_pixels[j], histogram[j], (j == SPLIT - 1) ? (IMG_HEIGHT / SPLIT - 1) : (IMG_HEIGHT / SPLIT + 1));
83     }
84
85     // Calculate the norm
86     uint16_t temp;
87     for (i = 0; i < HISTOGRAM_WIDTH; i++) {
88         loop_gather1: for (j = 0; j < SPLIT; j++) {
89             temp = (j == 0) ? histogram[0][i] : temp + histogram[j][i];
90         }
91         histogram[0][i] = temp;
92         histogram_product[i] = histogram[0][i] * histogram[0][i];
93         acum += histogram_product[i];
94     }
95     float norm = fast_sqrt(acum);
96
97     // Normalize the histogram
98     for (i = 0; i < HISTOGRAM_WIDTH; i++) {
99         feature[i] = histogram[0][i] / norm;
100     }
101 }

```

```

102
103 void Classifier::SVM(float features[HISTOGRAM_WIDTH], uint8_t *prediction) {
104
105     uint8_t k;
106     uint16_t i;
107     float predictors[HISTOGRAM_WIDTH];
108     float b;
109     float z[HISTOGRAM_WIDTH];
110     float acum[TYPES] = {0};
111     float predicted;
112     float last_predicted = -100000;
113
114     loop_histogram: for (i = 0; i < HISTOGRAM_WIDTH; i++){
115         loop_types1: for (k = 0; k < TYPES; k++){
116             predictors[i] = (features[i] - mu[k][i])/sigma[k][i];
117             z[i] = predictors[i]/scale[k];
118             acum[k] += z[i] * beta[k][i];
119         }
120     }
121
122     loop_types2: for (k = 0; k < TYPES; k++){
123         predicted = acum[k] + bias[k];
124
125         if (predicted > last_predicted)
126         {
127             *prediction = k;
128             last_predicted = predicted;
129         }
130     }
131 }
132
133 //Similar to Newton-Raphson
134 float Classifier::fast_sqrt(float number) {
135     union {
136         float f;
137         uint32_t i;
138     } conv = { number };
139
140     conv.i = 0x1fbd1df5 + (conv.i >> 1);
141     conv.f = 0.5f * (conv.f + number / conv.f);
142
143     return conv.f;
144 }

```

```
1  #ifndef TESTBENCH_H
2  #define TESTBENCH_H
3
4  #include <systemc.h>
5  #include "classifier.h"
6  #include <fstream>
7
8  #define IMG_WIDTH 28
9  #define IMG_HEIGHT 28
10
11 SC_MODULE(Testbench) {
12     sc_signal<bool> clk;
13     sc_signal<bool> reset;
14     sc_signal<bool> start;
15     sc_signal<uint8_t> pixels[IMG_WIDTH * IMG_HEIGHT];
16     sc_signal<uint8_t> prediction;
17
18     Classifier *classifier;
19
20     void run();
21     void load_test_image(const char *filename);
22
23     SC_CTOR(Testbench) {
24         classifier = new Classifier("classifier");
25         classifier->clk(clk);
26         classifier->reset(reset);
27         classifier->start(start);
28         for (int i = 0; i < IMG_WIDTH * IMG_HEIGHT; i++) {
29             classifier->pixels[i](pixels[i]);
30         }
31         classifier->prediction(prediction);
32
33         SC_THREAD(run);
34     }
35 };
36
37 #endif
```

```
1  //////////////////////////////////
2  // TESTBENCH.CPP //
3  //////////////////////////////////
4
5  #include "testbench.h"
6
7  // Thread for global execution
8  void Testbench::run() {
9      int test = 0;
10     reset = true;
11     start = false;
12     wait(10, SC_NS);
13
14     reset = false;
15     wait(10, SC_NS);
16
17     // Load an image
18     load_test_image("000000.raw");
19
20     start = true;
21     wait(10, SC_NS);
22     cout << "Prediction: " << (int)prediction.read() << endl;
23     start = false;
24
25     wait(100, SC_NS);
26
27     sc_stop();
28 }
29
30 // Intermediate memory
31 void Testbench::load_test_image(const char *filename) {
32
33     std::ifstream file(filename, std::ios::binary);
34
35     if (!file.is_open()) {
36         std::cerr << "Unable to open the file." << std::endl;
37         sc_stop();
38     }
39
40     uint8_t buffer[IMG_WIDTH * IMG_HEIGHT];
41     file.read(reinterpret_cast<char *>(buffer), IMG_WIDTH * IMG_HEIGHT); //
42     file.close();
43
44     for (int i = 0; i < IMG_WIDTH * IMG_HEIGHT; i++) {
45         pixels[i] = buffer[i];
46     }
47 }
```

```
1  //////////////////////////////////
2  //  MAIN.CPP  //
3  //////////////////////////////////
4
5  #include <systemc.h>
6  #include "testbench.h"
7
8  int sc_main(int argc, char* argv[]) {
9      Testbench tb("tb");
10
11      sc_start();
12
13      return 0;
14  }
```